Docker

Lernunterlage: Image und Container, Layer, Volumes, Multi-Stage-Build, warum nicht root, Secrets nicht ins Image

1 Worum es geht

Home Control läuft bisher als Dienst direkt auf dem PC. Mit Docker gibt es eine zweite Möglichkeit: Die App steckt mit allem, was sie braucht (.NET-Laufzeit, gebaute Oberfläche), in einem **Container**. Der startet auf jedem Rechner mit Docker gleich, egal welches .NET oder Node dort installiert ist. Der Dienst auf dem PC bleibt, wie er ist.

```
docker compose --profile demo up   # Demo, ohne Geheimnisse, http://localhost:5300
cp .env.example .env                # Betrieb: Werte eintragen
docker compose up -d                # Betrieb, http://localhost:5400
```

2 Image und Container

Begriff	Erklärung	Vergleich
Image	unveränderlicher Bauplan mit Dateien und Startbefehl, entsteht mit docker build	Klasse / Backform
Container	ein laufendes Exemplar eines Images, mit eigenem Dateisystem und Netz	Objekt / Kuchen
Dockerfile	Rezept, aus dem das Image gebaut wird	Quellcode
Volume	Speicher außerhalb des Containers, überlebt ihn	externe Festplatte
Compose	beschreibt mehrere Container, Ports, Volumes in einer YAML-Datei	Startskript

Ein Container ist zum **Wegwerfen** gedacht: Bei einem Update baut man ein neues Image und startet einen neuen Container. Alles, was bleiben soll (Datenbank, Schlüssel), gehört deshalb in ein Volume.

3 Layer und Cache

Jede Anweisung im Dockerfile erzeugt eine **Schicht** (Layer). Docker merkt sich die Schichten. Hat sich an einer Zeile und an allem davor nichts geändert, kommt die Schicht aus dem Cache. Deshalb die Reihenfolge:

```
COPY frontend/package.json frontend/package-lock.json ./
RUN npm ci                 # teuer, aber nur neu, wenn sich die Paketliste ändert
COPY frontend/ ./
RUN npm run build          # bei jeder Codeänderung neu, geht schnell
```

Würde man zuerst den ganzen Code kopieren, müsste `npm ci` bei jeder kleinen Änderung neu laufen. Beim Backend gilt dasselbe: erst die `.csproj` und `dotnet restore`, dann der Code.

4 Multi-Stage-Build

Stufe	Basis-Image	Ergebnis	Im fertigen Image?
1 frontend	node:22-alpine	frontend/dist (HTML, JS, CSS)	nur dist
2 backend	dotnet/sdk:10.0	/app (veröffentlichte DLLs)	nur /app
3 final	dotnet/aspnet:10.0	Laufzeit + /app + wwwroot	ja

- Nur die **letzte Stufe** wird zum Image. Node, das .NET SDK (1,3 GB) und der Quellcode bleiben zurück.
- Kleiner: 409 MB statt über 1,3 GB. Davon sind 342 MB das aspnet-Basis-Image, Home Control selbst ist der kleine Rest.
- Sicherer: Im Image gibt es keinen Compiler und kein npm, die ein Angreifer benutzen könnte.
- `COPY --from=backend /app ./` holt gezielt nur das Ergebnis aus einer früheren Stufe.

5 Volumes

Volume	Im Container	Inhalt	Warum eigenes Volume?
homecontrol-daten	/data	homecontrol.db	Daten überleben Updates
homecontrol-schluessel	/keys	Data-Protection-Schlüssel	sonst ist das Blink-Token nach einem Update unlesbar

Getestet: Benutzer angelegt, Container mit `--force-recreate` neu erzeugt, Anmeldung klappt weiter. Die Demo benutzt dagegen **tmpfs** (nur Arbeitsspeicher): Jeder Start beginnt mit dem frischen Beispielhaus.

Der Pfad der Schlüssel ist dafür einstellbar geworden: `HomeControl:DataProtectionKeysPath` (im Image `/keys`). Ohne Einstellung bleibt alles wie beim PC-Dienst.

6 Warum nicht als root?

- Standardmäßig läuft ein Prozess im Container als **root**. Findet ein Angreifer eine Lücke in der App, hätte er im Container alle Rechte und einen kürzeren Weg zum Rechner dahinter.
- `USER $APP_UID`: Das aspnet-Image bringt den Benutzer app (UID 1654) mit. Die App läuft mit dessen Rechten und darf nur in `/data` und `/keys` schreiben.
- Port 8080 statt 80: Ports unter 1024 dürfte nur root öffnen.
- Geprüft: `docker run --entrypoint id homecontrol` zeigt `uid=1654(app)`, die Schlüsseldatei im Volume hat Rechte 600.

7 Secrets nicht ins Image

Ein Image kann man weitergeben, hochladen, auf einen Server kopieren. Was darin steckt, ist für jeden lesbar, der das Image hat, auch alte Schichten. Deshalb:

- **Kein Geheimnis im Dockerfile:** kein `COPY .env`, kein Schlüssel in `ENV`.
- `.dockerignore` schließt `.env` und alle `*.db` aus: Sie gelangen nicht einmal in den Build-Kontext.
- Geheimnisse kommen erst beim **Start**: `env_file: .env` in docker-compose.yml.

- `.env.example` ist die Vorlage im Repo, **ohne Werte**. Die echte `.env` steht in .gitignore.
- Ein Test prüft das bei jedem `./pruefen.sh`: .env in .dockerignore und .gitignore, leere Werte in der Vorlage, `USER $APP_UID` im Dockerfile.

> Das Image ist der Bauplan und darf öffentlich sein. Geheimnisse gehören in die Umgebung beim Start, nie in den Bauplan.

8 Healthcheck und Profile

- **Healthcheck:** Docker fragt alle 30 Sekunden, ob die App gesund ist. Das aspnet-Image hat kein curl, deshalb prüft sich die App selbst: `dotnet HomeControl.Api.dll --healthcheck` fragt `/healthz` (läuft, Datenbank erreichbar) und endet mit 0 oder 1. `docker compose ps` zeigt dann „healthy“.
- **Profile:** `betrieb` (echter Dienst) und `demo`. In der .env steht `COMPOSE_PROFILES=betrieb`, deshalb startet `docker compose up` den Betrieb. `--profile demo` startet nur die Demo, ganz ohne .env, also auch auf dem Schul-PC.
- **Commit:** Im Container gibt es kein .git. Der Build-Parameter `GIT_COMMIT` wird zu `-p:SourceRevisionId`, die Einstellungsseite zeigt ihn an.

9 Kurzantworten für Rückfragen

Frage	Antwort
Unterschied Image und Container?	Image = Bauplan (unveränderlich), Container = laufendes Exemplar davon.
Was ist ein Layer?	Eine Schicht pro Anweisung im Dockerfile. Unveränderte Schichten kommen aus dem Cache.
Warum Multi-Stage?	Werkzeuge nur zum Bauen, im Image nur die Laufzeit: kleiner und sicherer.
Warum Volumes?	Container sind Wegwerfware. Datenbank und Schlüssel sollen Updates überleben.
Warum nicht root?	Eine Lücke in der App gibt dann nur die Rechte des Benutzers app, nicht alle.
Wo sind die Passwörter?	In .env, nur beim Start. Nie im Image, nie im Repo.
Ersetzt Docker den Dienst?	Nein, zusätzlich. Blink darf nur an einer Stelle laufen, deshalb im Container aus.
Wie weiß Docker, ob es läuft?	Healthcheck: die App fragt ihren eigenen Endpunkt /healthz ab.

10 Merksätze

> • Image = Bauplan, Container = laufendes Exemplar, Volume = was bleiben soll.
> • Selten Geändertes zuerst kopieren, dann nutzt Docker den Cache.
> • Multi-Stage: bauen mit dem SDK, ausliefern mit der Laufzeit.
> • Nie als root, nie Geheimnisse im Image.
> • Eine Funktion erst dokumentieren, wenn sie getestet ist (Start, Neustart, Sicherung).